UNIVERSIDADE LUTERANA DO BRASIL
EDUCAÇÃO A DISTÂNCIA

Curso Superior de Tecnologia em Análise e Desenvolvimento de Sistemas

PROJETO TECNOLÓGICO EM DESENVOLVIMENTO DE SISTEMAS

Etapa - Projeto

Plataforma Nexo - Tutor de Estudo Guiado

Gabriel Porto dos Santos

gpsnts@rede.ulbra.br

Sumário

1. Diagrama de Caso de Uso 2
Resumo dos casos de uso 2
Descrição dos casos de uso 4
UC01 — Autenticar-se 4
UC02 — Enviar material de estudo 4
UC03 — Informar tópico de estudo 5
UC04 — Indexar material (interno) 5
UC05 — Sugerir divisão em tópicos (interno) 5
UC06 — Gerar resumo 6
UC07 — Gerar questões 6
UC08 — Obter conteúdo-base (interno) 7
UC09 — Verificar fidelidade da questão (interno) 7
UC10 — Gerar prova por escopo 7
UC11 — Responder questões 8
UC12 — Atualizar agendamento de revisão (interno) 8
UC13 — Realizar revisão adaptativa 9
UC14 — Consultar painel de desempenho 9
UC15 — Organizar conteúdo por assunto 9
UC16 — Consultar histórico de materiais e tópicos 10
UC17 — Excluir material 10
UC18 — Excluir conta e dados 10
UC19 — Exportar dados de estudo 11
2. Diagrama Entidade-Relacionamento 11
Principais decisões de modelagem 11
Dicionário de dados 14
3. Arquitetura do Sistema 18
4. Ferramentas Previstas para o Desenvolvimento do Projeto 19
5. Cronograma 20

Lista de Figuras

Figura 1 – Diagrama de casos de uso (UC) da Plataforma Nexo (em azul, casos de uso internos) 3
Figura 2 – Isolamento por conta: todas as demais tabelas referenciam usuario por usuario_id e são protegidas por Row-Level Security 11
Figura 3 – Diagrama entidade-relacionamento (ER) da Plataforma Nexo 13
Figura 4 – Arquitetura da Plataforma Nexo: componentes e sentido das chamadas 19

1. Diagrama de Caso de Uso

O diagrama de casos de uso (UC) da Plataforma Nexo (Figura 1) representa as funções previstas para o sistema. O ator principal é o Usuário (estudante). O Provedor do modelo de linguagem (modelo de inteligência artificial, IA, que gera texto) é um ator secundário: participa da sugestão de divisão do material em tópicos, da obtenção do conteúdo-base e da verificação de fidelidade das questões. Os casos de uso estão agrupados em cinco blocos: Acesso, Entrada de estudo, Geração de conteúdo, Estudo e revisão e Gestão de conteúdo. Os casos de uso internos (em azul), executados pelo próprio sistema, são ligados por «include» (o caso de uso de origem sempre executa o incluído). O «extend» indica que a geração de provas pode, quando faltarem questões, acionar a geração de novas questões.

Nas tabelas e nas descrições, RF significa requisito funcional e RNF significa requisito não funcional, ambos conforme o Documento de Requisitos entregue na etapa anterior. Os requisitos funcionais RF01 a RF19 estão todos cobertos pelos casos de uso; os UC17 e UC18 atendem ao requisito não funcional de retenção e privacidade de dados (Lei Geral de Proteção de Dados, LGPD), e o UC19, ao de portabilidade de dados.

Resumo dos casos de uso

ID	Caso de uso	Ator(es)	Requisitos
UC01	Autenticar-se	Usuário	RF01
UC02	Enviar material de estudo	Usuário	RF02, RF04 (via UC04), RF10 (via UC05)
UC03	Informar tópico de estudo	Usuário	RF03
UC04	Indexar material	Sistema (interno)	RF04
UC05	Sugerir divisão em tópicos	Provedor do modelo de linguagem	RF10, RF16
UC06	Gerar resumo	Usuário	RF05
UC07	Gerar questões	Usuário	RF06, RF09 (via UC09)
UC08	Obter conteúdo-base	Provedor do modelo de linguagem	RF07, RF08
UC09	Verificar fidelidade da questão	Provedor do modelo de linguagem	RF09
UC10	Gerar prova por escopo	Usuário	RF15, RF16, RF17, RF18
UC11	Responder questões	Usuário	RF11
UC12	Atualizar agendamento de revisão	Sistema (interno)	RF12
UC13	Realizar revisão adaptativa	Usuário	RF12, RF13
UC14	Consultar painel de desempenho	Usuário	RF14
UC15	Organizar conteúdo por assunto	Usuário	RF10
UC16	Consultar histórico de materiais e tópicos	Usuário	RF19
UC17	Excluir material	Usuário	RNF de retenção e privacidade de dados (LGPD)
UC18	Excluir conta e dados	Usuário	RNF de retenção e privacidade de dados (LGPD)
UC19	Exportar dados de estudo	Usuário	RNF de portabilidade de dados

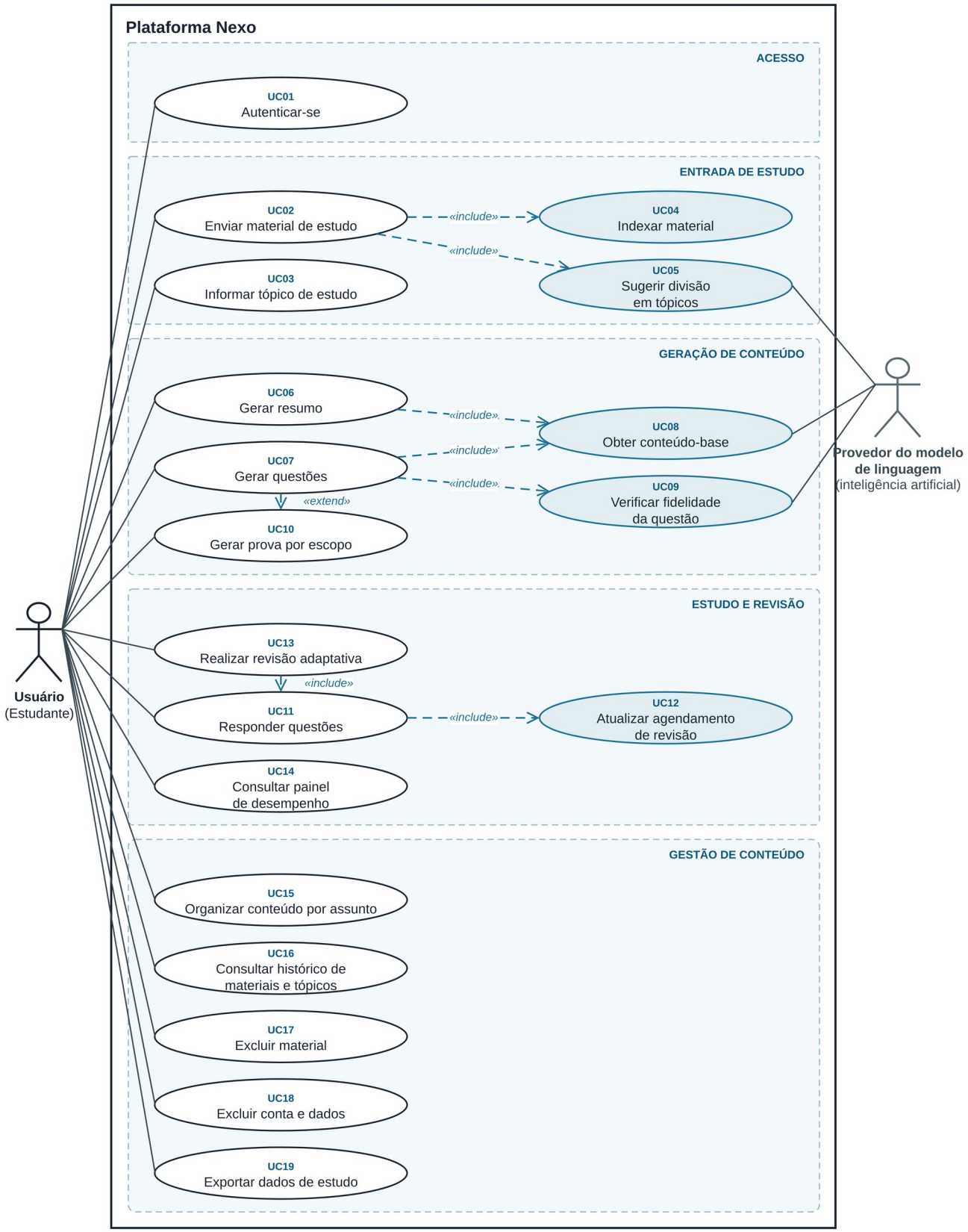


Figura 1 – Diagrama de casos de uso (UC) da Plataforma Nexo (em azul, casos de uso internos)

Descrição dos casos de uso

Para cada caso de uso, são apresentados o(s) ator(es), a descrição de como ele funciona, a pré-condição, o fluxo principal, os fluxos alternativos (A1, A2…), a pós-condição e os requisitos relacionados, conforme o Documento de Requisitos.

UC01 — Autenticar-se

Ator(es): Usuário **Requisitos:** RF01

Descrição: Permite criar uma conta ou entrar com e-mail e senha. Todo o conteúdo do estudante fica isolado na sua conta.

Pré-condição: Nenhuma (ponto de entrada do sistema).

Fluxo principal:

1. Usuário acessa a tela de login.
2. Usuário informa e-mail e senha.
3. Sistema valida as credenciais.
4. Sistema inicia a sessão e exibe a tela inicial.

Fluxos alternativos:

A1. Credenciais inválidas: o sistema exibe mensagem de erro e permite nova tentativa.
A2. Primeiro acesso: o usuário escolhe "Criar conta" e informa nome, e-mail e senha; o sistema cria a conta (a senha é armazenada apenas como hash, nunca em texto puro) e o assunto padrão "Geral", e inicia a sessão.
A3. E-mail já cadastrado: o sistema informa e sugere entrar com a conta existente.

Pós-condição: Sessão autenticada, com acesso às demais funcionalidades.

UC02 — Enviar material de estudo

Ator(es): Usuário **Requisitos:** RF02, RF04 (via UC04), RF10 (via UC05)

Descrição: O estudante envia um material próprio para que resumos e questões sejam gerados com base especificamente nele. O material é indexado e dividido em tópicos, o que permite estudar e ser avaliado por partes do mesmo material. Um tópico também pode reunir partes de mais de um material.

Pré-condição: Usuário autenticado (UC01).

Fluxo principal:

1. Usuário acessa "Novo estudo" e escolhe "Enviar material".
2. Usuário seleciona o arquivo e, opcionalmente, o assunto de destino.
3. Sistema valida o formato (aceitos: PDF, DOCX e TXT), o tamanho (máximo de 20 megabytes), o número de páginas (máximo de 50) e se o arquivo é repetido (comparando o hash, código calculado a partir do conteúdo do arquivo).
4. Sistema grava o material com status "pendente" e executa "Indexar material" (UC04).
5. Sistema executa "Sugerir divisão em tópicos" (UC05) e apresenta a proposta.
6. Usuário revisa a proposta (renomeia, une, divide ou ajusta os limites dos tópicos) e confirma.
7. Sistema cria os tópicos e suas fontes (a faixa de trechos de cada tópico neste material) no assunto escolhido (ou em "Geral") e informa que o material está pronto.

Fluxos alternativos:

A1. Formato, tamanho ou número de páginas fora dos limites: o sistema informa os limites aceitos e solicita novo envio.
A2. Arquivo repetido (mesmo hash de um material do usuário cujo status não é "falhou"): o sistema informa e oferece abrir o material existente.
A3. Usuário recusa a divisão proposta: o material inteiro vira um único tópico.
A4. Falha no processamento (ex.: arquivo sem texto extraível): o status do material passa a "falhou", o sistema informa o motivo e permite novo envio; no reenvio, o registro anterior com status "falhou" é removido (os registros das operações desse material são mantidos, sem a referência a ele).
A5. Usuário associa a faixa de trechos a um tópico já existente, em vez de criar um novo: o sistema cria a fonte desse material no tópico, que passa a ter mais de um material, respeitando a regra de não sobreposição das faixas.

Pós-condição: Material com status "pronto", indexado e dividido em tópicos.

UC03 — Informar tópico de estudo

Ator(es): Usuário **Requisitos:** RF03

Descrição: Sem material em mãos, o estudante digita livremente o tópico que quer estudar (ex.: "Movimento Retilíneo Uniforme"). O conteúdo é gerado a partir do conhecimento geral do modelo de linguagem.

Pré-condição: Usuário autenticado (UC01).

Fluxo principal:

1. Usuário acessa "Novo estudo" e escolhe "Informar tópico".
2. Usuário digita o nome do tópico e, opcionalmente, escolhe o assunto.
3. Sistema registra o tópico ao final do assunto escolhido (ou de "Geral"), sem material associado.

Fluxos alternativos:

A1. Campo vazio: o sistema impede o envio e solicita o nome do tópico.

Pós-condição: Tópico registrado e disponível para geração de resumo e questões.

UC04 — Indexar material (interno)

Ator(es): — (caso de uso interno, incluído por UC02) **Requisitos:** RF04

Descrição: Prepara o material para consulta por similaridade. O serviço de processamento (em Python) extrai o texto, as tabelas e as figuras, divide em trechos e gera os embeddings (vetores numéricos que representam o significado de cada trecho) e devolve o resultado ao backend, que é o único componente que grava no banco de dados (PostgreSQL com a extensão pgvector, usada para busca vetorial).

Pré-condição: Arquivo válido recebido em UC02, com material em status "pendente".

Fluxo principal:

1. Backend altera o status do material para "processando", cria o registro da operação indexacao_material, ligado ao material (status pendente, depois em_andamento) e envia o arquivo ao serviço de processamento. Durante todo o processamento, o backend atualiza o percentual de andamento do registro (extração, divisão, embeddings em lotes e gravação), que o frontend consulta para exibir uma barra de progresso.
2. Serviço extrai o texto, as tabelas e as figuras, registrando o número da página de cada parte (quando o formato tem páginas). Cada tabela é convertida em texto; cada figura é descrita por um modelo de linguagem com visão, até o limite de figuras por material (padrão: 30, valor configurável); as figuras além do limite são ignoradas e o usuário é informado.
3. Serviço divide o conteúdo em trechos, respeitando a estrutura (parágrafos, seções, tabelas e figuras), classifica cada trecho como texto, tabela ou figura e gera o embedding de cada um.
4. Backend grava os trechos, os embeddings e o nome do modelo de embedding utilizado.
5. Backend soma tokens e custo estimado ao registro, marca-o como concluida (com finalizada_em e andamento em 100%) e altera o status do material para "pronto".

Fluxos alternativos:

A1. Material sem texto extraível (ex.: PDF escaneado, que contém apenas imagens): o status do material passa a "falhou", o registro da operação passa a falhou e o usuário é informado.

A2. Modelo de embedding substituído: o sistema reindexa o material atualizando embedding e modelo_embedding de cada trecho no lugar; os trechos não são apagados nem recriados, porque as faixas dos tópicos dependem da ordem deles.

A3. Falha ao descrever uma figura (modelo de linguagem indisponível ou erro): a figura é ignorada, o processamento continua e o usuário é informado de quantas figuras ficaram de fora.

Pós-condição: Material consultável por busca de similaridade.

UC05 — Sugerir divisão em tópicos (interno)

Ator(es): Provedor do modelo de linguagem (ator secundário) — incluído por UC02 **Requisitos:** RF10, RF16

Descrição: Propõe como dividir o material em tópicos (ex.: capítulos ou seções), indicando o primeiro e o último trecho de cada um. É isso que permite ao estudante delimitar "até onde estudou" dentro de um mesmo material.

Pré-condição: Material indexado (UC04).

Fluxo principal:

1. Sistema cria o registro da operação sugestao_topicos, ligado ao material, e identifica a estrutura do material (títulos, seções e páginas).

2. Sistema pede ao modelo de linguagem uma proposta de tópicos, com nome e limites (ordem do trecho inicial e do trecho final).
3. Sistema valida que os tópicos cobrem todo o material, sem lacunas e sem sobreposição.
4. Sistema marca o registro como concluida e devolve a proposta ao UC02.

Fluxos alternativos:

A1. Material sem estrutura clara ou falha no modelo de linguagem: o sistema propõe a divisão em blocos de tamanho semelhante.

Pós-condição: Proposta de tópicos com limites, disponível para confirmação.

UC06 — Gerar resumo

Ator(es): Usuário **Requisitos:** RF05

Descrição: Gera um resumo do conteúdo estudado, a partir do material enviado ou do tópico informado.

Pré-condição: Existe um material (UC02) ou tópico (UC03).

Fluxo principal:

1. Usuário seleciona o material ou tópico e escolhe "Gerar resumo".
2. Sistema cria o registro da operação resumo (status pendente, depois em_andamento).
3. Sistema executa o caso de uso incluído "Obter conteúdo-base" (UC08).
4. Sistema gera o resumo a partir do conteúdo-base.
5. Sistema grava o resumo (cada nova geração cria um novo resumo), marca o registro como concluida e exibe o resumo.

Fluxos alternativos:

A1. Falha ou indisponibilidade do serviço do modelo de linguagem: o registro passa a falhou, o sistema exibe mensagem clara e permite nova tentativa.
A2. Limite de requisições por usuário atingido (padrão: 30 operações por hora, valor configurável): o sistema informa e orienta a aguardar.
A3. Limite de gasto por usuário atingido (valor e janela configuráveis): o sistema informa, não inicia a operação e orienta a aguardar a próxima janela.

Pós-condição: Resumo disponível para consulta.

UC07 — Gerar questões

Ator(es): Usuário **Requisitos:** RF06, RF09 (via UC09)

Descrição: Gera questões de múltipla escolha sobre o conteúdo estudado. Cada questão passa por verificação de fidelidade e de repetição antes de ser gravada e exibida. Também pode ser acionado pela geração de provas quando faltarem questões (extensão).

Pré-condição: Existe um material (UC02) ou tópico (UC03).

Fluxo principal:

1. Usuário seleciona o material ou tópico, informa a quantidade e escolhe "Gerar questões".
2. Sistema cria o registro da operação questoes (status pendente, depois em_andamento) e atualiza o percentual de andamento a cada questão concluída.
3. Sistema executa "Obter conteúdo-base" (UC08).
4. Sistema gera as questões.
5. Sistema executa "Verificar fidelidade da questão" (UC09) para cada questão.
6. Sistema verifica se cada questão aprovada é repetida: calcula o hash do enunciado normalizado e compara o embedding do enunciado com o das questões já gravadas do mesmo tópico; se o hash for igual ou a similaridade atingir o limiar configurado (padrão sugerido: 0,90), a questão é descartada.
7. Sistema grava as questões aprovadas e não repetidas (com o hash e o embedding do enunciado), marca o registro como concluida e as apresenta.

Fluxos alternativos:

A1. Questão reprovada na verificação: é descartada e uma nova é gerada, até o limite de tentativas do UC09.
A2. Falha no modelo de linguagem, limite de requisições ou limite de gasto: mesma tratativa do UC06.
A3. Questão repetida: é descartada, contabilizada no registro (questões repetidas) e uma nova é gerada, com o mesmo limite de tentativas por questão do UC09 (padrão: 3).

Pós-condição: Questões disponíveis para serem respondidas (UC11).

UC08 — Obter conteúdo-base (interno)

Ator(es): Provedor do modelo de linguagem (ator secundário) — incluído por UC06 e UC07 **Requisitos:** RF07, RF08

Descrição: Define a estratégia de geração. Com material, busca por similaridade os trechos mais relevantes (técnica de geração aumentada por recuperação, RAG), limitados às fontes do tópico (cada material e a parte dele que pertence ao tópico), e os usa como base, mantendo fidelidade à fonte. Sem material, usa o conhecimento geral do modelo sobre o tópico.

Pré-condição: Solicitação vinda de UC06 ou UC07, com material ou tópico associado.

Fluxo principal:

1. Sistema verifica se há material associado.
2. Com material: busca, apenas dentro das fontes do tópico (em cada material, a faixa de trechos do tópico; para a geração pedida sobre o material inteiro, todo o material), os trechos mais similares, reúne os mais similares entre as fontes e os envia ao modelo como contexto.
3. Sem material: solicita ao modelo o conteúdo geral sobre o tópico.
4. Sistema soma os tokens e o custo estimado ao registro da operação que o chamou (UC06 ou UC07).
5. Sistema devolve o conteúdo-base (e a lista dos trechos usados) ao caso de uso solicitante.

Fluxos alternativos:

A1. Nenhum trecho suficientemente relevante: o sistema informa e sugere reformular o tópico ou enviar outro material.
A2. Embeddings gerados por outro modelo: o sistema reindexa o material (UC04) antes da busca.
A3. Falha no modelo de linguagem: o erro é repassado ao caso de uso solicitante, que marca o registro da sua operação como falhou.

Pós-condição: Conteúdo-base disponível; tokens e custo somados ao registro da operação que chamou.

UC09 — Verificar fidelidade da questão (interno)

Ator(es): Provedor do modelo de linguagem (ator secundário) — incluído por UC07 **Requisitos:** RF09

Descrição: Confere se cada questão gerada é sustentada pelo conteúdo de origem, descartando as que não atendem. Sem material, avalia a consistência interna da questão.

Pré-condição: Questão gerada em UC07.

Fluxo principal:

1. Sistema cria o registro da operação verificacao_questao (status pendente, depois em_andamento) no início da verificação de uma geração.
2. Para cada questão, o sistema pergunta ao modelo se a resposta correta é sustentada pelos trechos usados na geração.
3. Aprovada: a questão é gravada e ligada aos trechos que a sustentam.
4. Reprovada: é descartada (não é gravada) e uma nova geração é solicitada, até 3 tentativas por questão (valor padrão, configurável).
5. Ao terminar, o sistema grava no registro as quantidades de questões aprovadas e rejeitadas e o marca como concluida.

Fluxos alternativos:

A1. Tentativas esgotadas: a questão não é exibida e a rejeição é contabilizada.

Pós-condição: Apenas questões aprovadas são gravadas; a taxa de rejeição (rejeitadas divididas pelo total de aprovadas e rejeitadas) fica disponível.

UC10 — Gerar prova por escopo

Ator(es): Usuário **Requisitos:** RF15, RF16, RF17, RF18

Descrição: Monta uma prova escolhendo os assuntos, o ponto de estudo já alcançado em cada um, a quantidade de questões e se os assuntos serão misturados ou separados.

Pré-condição: Existe ao menos um assunto com tópicos e questões.

Fluxo principal:

1. Usuário escolhe "Gerar prova" e seleciona um ou mais assuntos.
2. Para cada assunto, define o tópico-limite, isto é, até qual tópico já estudou (sem limite, entram todos os tópicos do assunto).
3. Usuário informa a quantidade de questões.

4. Usuário escolhe uma prova única, com assuntos misturados, ou provas separadas por assunto.
5. Sistema divide a quantidade pedida igualmente entre os assuntos (as questões que sobrarem da divisão vão, uma a uma, aos assuntos na ordem escolhida) e sorteia, em cada assunto, questões dos tópicos cuja posição é menor ou igual à do tópico-limite; se um assunto tiver menos questões do que a sua cota, a sobra é redistribuída entre os demais.
6. Sistema sorteia a ordem das questões, grava a(s) prova(s) com status "gerada" e as apresenta. Em provas separadas, cada prova recebe a quantidade total informada, e as provas do mesmo pedido recebem o mesmo identificador de grupo e são apresentadas juntas.

Fluxos alternativos:

A1. Questões insuficientes (soma das questões elegíveis menor que a quantidade pedida): o sistema oferece gerar novas questões (UC07, extensão) ou reduzir a quantidade.

Pós-condição: Prova(s) criada(s), pronta(s) para serem respondidas em UC11.

UC11 — Responder questões

Ator(es): Usuário **Requisitos:** RF11

Descrição: O estudante responde a questões avulsas, de uma prova ou de uma sessão de revisão. Cada resposta é registrada e o sistema informa o acerto e o gabarito.

Pré-condição: Existem questões geradas (UC07), uma prova (UC10) ou uma sessão de revisão (UC13).

Fluxo principal:

1. Sistema apresenta uma questão.
2. Usuário seleciona uma alternativa.
3. Sistema confere a resposta e mostra o acerto ou erro e a alternativa correta.
4. Sistema registra a resposta (alternativa escolhida, acerto, data e hora e a prova, quando houver).
5. Sistema executa "Atualizar agendamento de revisão" (UC12).
6. Repete até o fim da sessão e exibe o resultado; se for uma prova, altera o status para "em andamento" na primeira resposta e para "concluída" (gravando a data e hora de conclusão) ao responder a última questão.

Fluxos alternativos:

A1. Abandono da sessão: as respostas já registradas são mantidas e a prova permanece "em andamento".

Pós-condição: Respostas registradas e agendamento de revisão atualizado.

UC12 — Atualizar agendamento de revisão (interno)

Ator(es): — (caso de uso interno, incluído por UC11) **Requisitos:** RF12

Descrição: Reagenda a próxima revisão da questão respondida, com uma variação do algoritmo SM-2 (método de repetição espaçada) adaptada a respostas binárias (certo ou errado). Ao contrário do SM-2 original, o fator de facilidade também diminui quando a resposta é incorreta. O agendamento é por questão; a prioridade do tópico é calculada em UC13. As datas usam o fuso horário America/Sao_Paulo.

Pré-condição: Resposta registrada em UC11.

Fluxo principal:

1. Sistema localiza o agendamento da questão para o usuário; se não existir (primeira resposta), cria-o com repetições 0, intervalo de 1 dia e fator de facilidade 2,50.
2. Resposta correta dada antes da data da próxima revisão, e que não é a primeira: o agendamento não é alterado.
3. Demais respostas corretas: soma 1 às repetições; o intervalo passa a 1 dia na 1ª repetição, 6 dias na 2ª e, da 3ª em diante, o intervalo anterior multiplicado pelo fator de facilidade anterior à atualização, arredondado para o inteiro mais próximo (metade para cima) e limitado a 365 dias; depois, o fator aumenta 0,10 (máximo 2,50).
4. Resposta incorreta (sempre reagenda): zera as repetições, define o intervalo em 1 dia e reduz o fator em 0,20 (mínimo 1,30).
5. Sistema grava a data da última revisão (hoje) e a da próxima (hoje mais o intervalo).

Fluxos alternativos:

A1. Primeira resposta da questão: o agendamento é criado no passo 1 e as regras dos passos 3 e 4 valem a partir dele.

Pós-condição: Agendamento da questão atualizado.

UC13 — Realizar revisão adaptativa

Ator(es): Usuário **Requisitos:** RF12, RF13

Descrição: Inicia uma sessão de revisão com os tópicos priorizados pelo sistema. A prioridade do tópico é calculada na consulta: primeiro vêm os tópicos com mais questões com revisão vencida (data da próxima revisão menor ou igual a hoje) e, em caso de empate, os de maior taxa de erro (respostas incorretas divididas pelo total de respostas do tópico).

Pré-condição: Existem questões com histórico de respostas.

Fluxo principal:

1. Usuário escolhe "Revisar".
2. Sistema calcula a prioridade de cada tópico e os lista em ordem decrescente.
3. Sistema monta a sessão com até 20 questões vencidas (valor padrão, configurável), começando pelos tópicos mais prioritários.
4. Usuário responde às questões (inclui UC11).
5. Sistema exibe o resumo da sessão.

Fluxos alternativos:

A1. Nenhuma questão com revisão vencida: o sistema informa e sugere estudar novo conteúdo.

Pós-condição: Sessão concluída; agendamentos atualizados.

UC14 — Consultar painel de desempenho

Ator(es): Usuário **Requisitos:** RF14

Descrição: Exibe a evolução do desempenho por assunto ao longo do tempo, destacando pontos fortes e fracos.

Pré-condição: Existe ao menos uma resposta registrada.

Fluxo principal:

1. Usuário acessa "Desempenho".
2. Sistema consulta o histórico de respostas.
3. Sistema calcula, por assunto e por tópico, o percentual de acertos em cada semana (de segunda a domingo).
4. Sistema exibe gráficos de evolução e a lista dos tópicos com menor percentual de acertos.

Fluxos alternativos:

A1. Dados insuficientes: o sistema informa que ainda não há histórico para exibir.

Pós-condição: Usuário identifica o que precisa revisar.

UC15 — Organizar conteúdo por assunto

Ator(es): Usuário **Requisitos:** RF10

Descrição: Cria assuntos (matérias), agrupa neles os tópicos e define a ordem entre eles. Essa organização é a base do escopo das provas.

Pré-condição: Existe ao menos um tópico (todo tópico pertence a um assunto; o padrão é "Geral").

Fluxo principal:

1. Usuário acessa "Meus assuntos".
2. Usuário cria um assunto ou seleciona um existente.
3. Usuário move tópicos para o assunto e define a ordem entre eles.
4. Sistema salva a organização.

Fluxos alternativos:

A1. Nome de assunto já existente: o sistema sugere reutilizar o assunto em vez de duplicá-lo.
A2. Tentativa de remover o assunto padrão "Geral": o sistema não permite.
A3. Tópico usado como tópico-limite de alguma prova: o sistema não permite movê-lo para outro assunto e informa as provas afetadas.
A4. Usuário adiciona ou remove fontes de um tópico (partes de materiais): o sistema valida que a faixa não se sobrepõe a outra já usada do mesmo material e salva; um tópico sem fontes passa a ser um tópico livre.

Pós-condição: Conteúdo organizado por assunto, disponível para UC10.

UC16 — Consultar histórico de materiais e tópicos

Ator(es): Usuário **Requisitos:** RF19

Descrição: Lista os materiais e tópicos já cadastrados, permitindo reutilizá-los em novas gerações de conteúdo ou provas.

Pré-condição: Usuário autenticado (UC01).

Fluxo principal:

1. Usuário acessa "Histórico".
2. Sistema lista materiais e tópicos por data e assunto.
3. Usuário abre um item para ver resumo e questões existentes ou gerar novo conteúdo.

Fluxos alternativos:

A1. Histórico vazio: o sistema orienta a iniciar um novo estudo.

Pós-condição: Item reaproveitado em nova geração ou prova.

UC17 — Excluir material

Ator(es): Usuário **Requisitos:** RNF de retenção e privacidade de dados (Lei Geral de Proteção de Dados, LGPD)

Descrição: Remove definitivamente um material enviado, com o arquivo, seus trechos e embeddings. Tópicos, questões, provas e registros de operações já criados são mantidos, sem a referência ao material (um tópico que tem outros materiais continua alimentado por eles).

Pré-condição: Material existente na conta do usuário.

Fluxo principal:

1. Usuário seleciona o material e escolhe "Excluir".
2. Sistema pede confirmação, informando que a ação é irreversível.
3. Usuário confirma.
4. Em uma única transação, o backend anula nos registros de operação a referência ao material e exclui o material (as fontes dos tópicos, os trechos e as ligações das questões com esses trechos são excluídos em cascata).
5. Após a confirmação da transação, o backend remove o arquivo do armazenamento (se a remoção falhar, é repetida).
6. Sistema confirma a exclusão.

Fluxos alternativos:

A1. Usuário cancela a confirmação: nada é alterado.

Pós-condição: Dados do material removidos da plataforma.

UC18 — Excluir conta e dados

Ator(es): Usuário **Requisitos:** RNF de retenção e privacidade de dados (Lei Geral de Proteção de Dados, LGPD)

Descrição: Remove definitivamente a conta e todos os dados do usuário: materiais e arquivos, trechos, assuntos, tópicos, questões, provas, respostas, agendamentos e registros de operações.

Pré-condição: Usuário autenticado (UC01).

Fluxo principal:

1. Usuário escolhe "Excluir conta".
2. Sistema informa que a ação é irreversível e pede a senha.
3. Usuário confirma com a senha.
4. Sistema remove os arquivos e exclui o usuário; todos os registros ligados a ele são excluídos em cascata.
5. Sistema encerra a sessão e confirma a exclusão.

Fluxos alternativos:

A1. Senha incorreta ou cancelamento: nada é alterado.

Pós-condição: Conta e dados removidos da plataforma.

UC19 — Exportar dados de estudo

Ator(es): Usuário **Requisitos:** RNF de portabilidade de dados

Descrição: Exporta o histórico de desempenho e as questões geradas em arquivo CSV ou JSON (formatos abertos de dados), evitando dependência exclusiva da plataforma.

Pré-condição: Usuário autenticado (UC01).

Fluxo principal:

1. Usuário escolhe "Exportar dados".
2. Usuário seleciona o conteúdo (desempenho, questões) e o formato.
3. Sistema gera o arquivo.
4. Sistema disponibiliza o download.

Fluxos alternativos:

A1. Sem dados para exportar: o sistema informa o usuário.

Pós-condição: Arquivo de exportação entregue ao usuário.

2. Diagrama Entidade-Relacionamento

A Figura 3 apresenta o modelo entidade-relacionamento (ER) do banco de dados da Plataforma Nexo, que usa PostgreSQL com a extensão pgvector (busca por similaridade de vetores). O modelo reúne 15 entidades em quatro grupos: conteúdo (material, trecho, assunto, tópico, topico_fonte, resumo, questão e questao_trecho); estudo e revisão (resposta e progresso_questao); provas (prova, prova_assunto e prova_questao); e controle (usuario e registro_geracao). Cada linha do diagrama é uma chave estrangeira, e o rótulo da linha indica a coluna que a implementa. A entidade usuario aparece sem ligações porque todas as demais tabelas a referenciam por usuario_id; essas ligações não foram desenhadas na Figura 3, para manter o diagrama legível, e são mostradas em separado na Figura 2. As siglas do diagrama estão explicadas na legenda da própria figura: PK é chave primária, FK é chave estrangeira e "(opcional)" indica coluna que aceita valor nulo. Os tipos, as restrições e os valores permitidos de cada coluna estão no dicionário de dados, mais adiante.

Principais decisões de modelagem

- Isolamento por conta (Lei Geral de Proteção de Dados, LGPD): todas as tabelas, exceto usuario, têm a coluna usuario_id. As chaves estrangeiras entre tabelas de dados são, em regra, compostas com usuario_id, o que impede um registro de referenciar dados de outro usuário (as três exceções estão no dicionário de dados). O PostgreSQL aplica Row-Level Security (segurança em nível de linha) em todas elas: a aplicação conecta com o papel nexo_app, que não é dono das tabelas, e define o usuário em cada transação com SET LOCAL app.usuario_id. A Figura 2 resume essa relação.

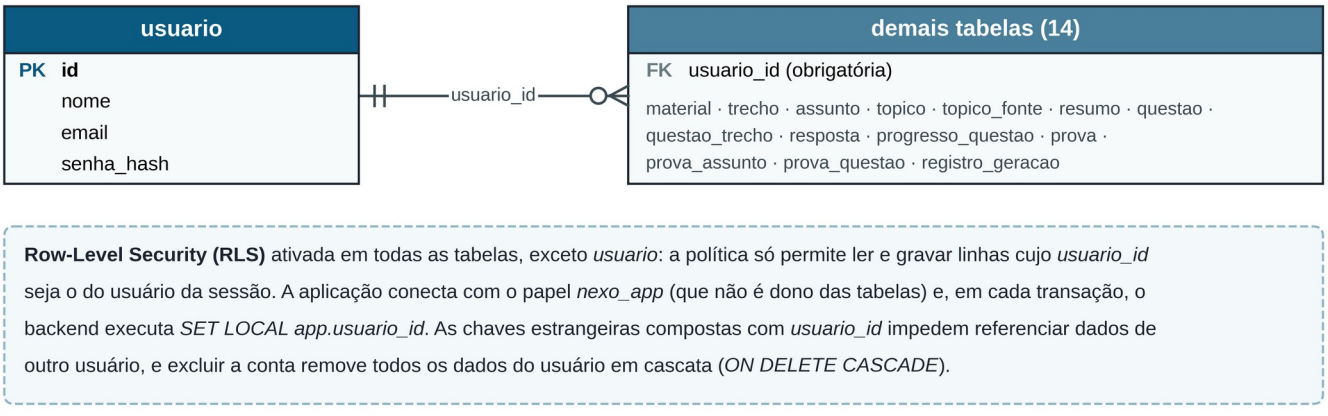


Figura 2 – Isolamento por conta: todas as demais tabelas referenciam usuario por usuario_id e são protegidas por Row-Level Security

- Escopo dentro do material (RF16): topico_fonte.ordem_trecho_inicial e ordem_trecho_final delimitam a parte de cada material que um tópico representa (proposta no UC05); um tópico pode ter várias fontes, de

materiais diferentes, no máximo uma por material. São chaves estrangeiras para trecho, o que impede apontar para trechos inexistentes, e uma restrição de exclusão impede que as faixas do mesmo material se sobreponham, mesmo entre tópicos diferentes. prova_assunto.topico_limite_id indica até qual tópico do assunto o estudante já estudou; a chave estrangeira composta (topico_limite_id, assunto_id) garante que o limite pertence ao assunto.

- Reordenação e ajuste de tópicos: a ordem dos tópicos, a ordem das questões da prova e as faixas de trechos têm restrições de unicidade ou de exclusão conferidas apenas no fim da transação. Isso permite trocar, deslocar, unir e dividir tópicos (UC02 e UC15) em várias instruções dentro da mesma transação.
- Embeddings: trecho.embedding é gravado junto com trecho.modelo_embedding. A coluna tem dimensão fixa, definida pelo modelo adotado (por exemplo, 1536 números, cerca de 6 quilobytes por trecho); trocar para um modelo de outra dimensão exige migração do esquema e reindexação do material (UC04), que atualiza os embeddings no lugar, sem recriar os trechos (os embeddings dos enunciados das questões, em questao.embedding, também são reindexados). A busca é exata, sempre dentro de um único material (algumas centenas de trechos), com índice comum em material_id. Um índice aproximado de vetores não é usado porque, ao filtrar por material, poderia devolver menos resultados que o pedido.
- Fontes das questões: a tabela questao_trecho liga cada questão a todos os trechos que a sustentam (e não a um só), pois a geração usa vários trechos como contexto.
- Revisão adaptativa (RF12 e RF13): o agendamento é por questão (progresso_questao), como no algoritmo SM-2; guardar o intervalo no tópico faria o intervalo crescer a cada resposta da mesma sessão. As regras do reagendamento estão no UC12. A prioridade do tópico é calculada na consulta (UC13): primeiro as questões com revisão vencida, depois a taxa de erro. A resposta guarda a alternativa escolhida, o que permite reexibir a prova corrigida, e só pode apontar para prova cuja lista de questões a contenha.
- Exclusões: ao excluir um material, o backend, em uma só transação, anula em registro_geracao a coluna material_id e exclui o material; as fontes dos tópicos (topico_fonte), os trechos e as ligações em questao_trecho são excluídos em cascata, e tópicos, questões, provas, respostas e registros de operação são mantidos. Ao excluir a conta, todos os dados do usuário são excluídos em cascata.
- Operações demoradas: indexar um material e gerar conteúdo demoram. O material tem status_processamento e cada operação tem um registro em registro_geracao com status (pendente, em_andamento, concluida ou falhou), usado para exibir o andamento ao usuário. Esse registro também guarda modelo, tokens, custo estimado e questões aprovadas e rejeitadas e, nas operações que ainda não têm tópico (indexacao_material e sugestao_topicos), o material a que se refere (material_id), e o percentual de andamento (progresso_percentual), consultado pelo frontend para exibir a barra de progresso, e serve de base ao limite de requisições por usuário.
- Questões repetidas: questao guarda enunciado_hash (SHA-256 do enunciado normalizado) e o embedding do enunciado. O banco impede enunciado idêntico no mesmo tópico (restrição de unicidade) e o backend descarta, antes de gravar, a questão de enunciado semelhante (similaridade de cosseno acima de um limiar configurável) a outra do mesmo tópico; o descarte é contabilizado em registro_geracao.questoes_repetidas.
- Provas separadas do mesmo pedido: prova.grupo_id recebe o mesmo valor (UUID gerado pelo backend) nas provas separadas geradas em um pedido e fica nulo na prova única; não há tabela própria para o grupo.
- Tabelas e figuras: trecho.tipo_conteudo distingue texto, tabela e figura. A tabela é gravada em formato de texto e a figura por uma descrição gerada por modelo de linguagem com visão (a imagem não é armazenada); assim, ambas entram na busca por similaridade e nas faixas dos tópicos.

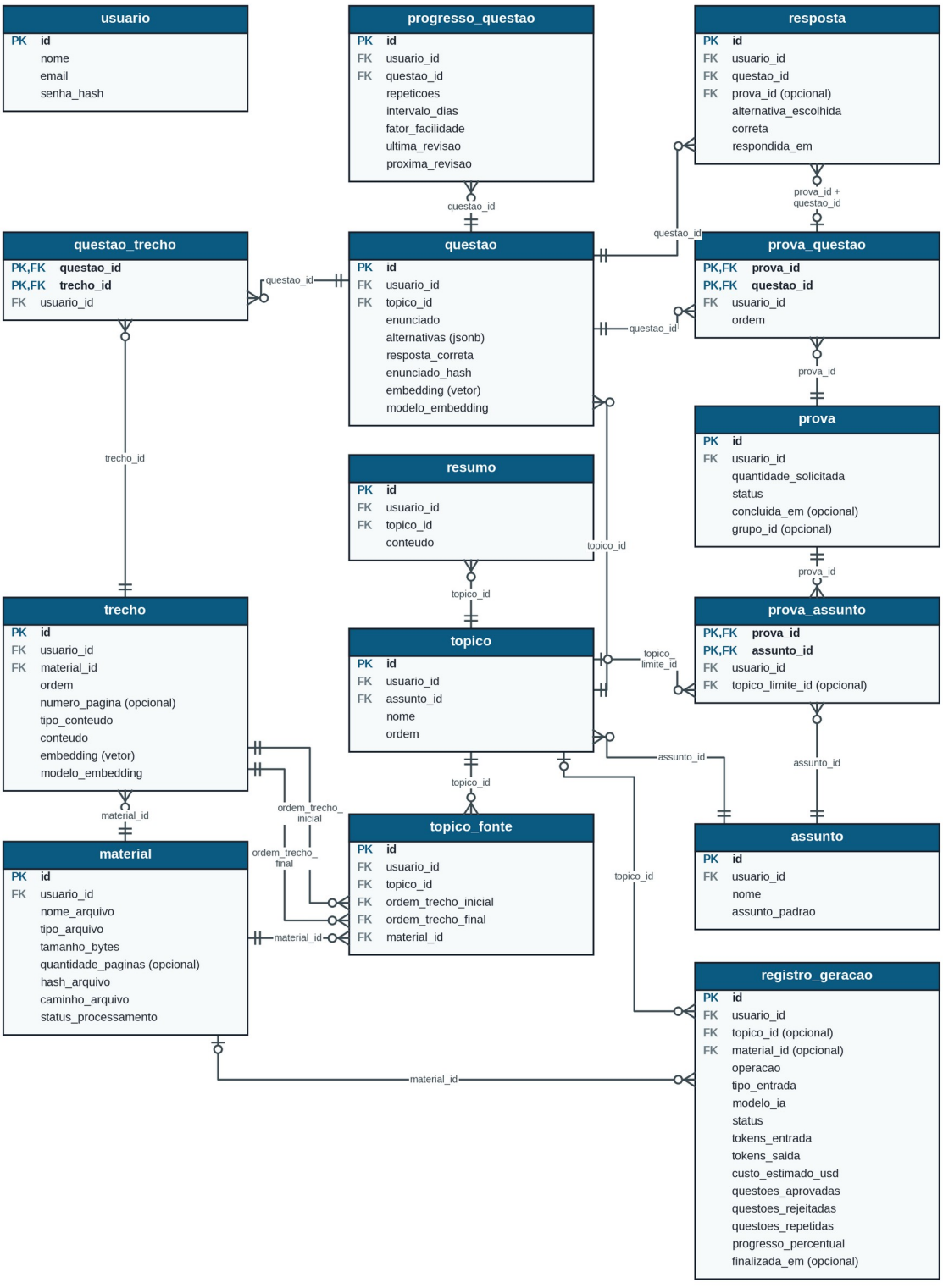


Figura 3 – Diagrama entidade-relacionamento (ER) da Plataforma Nexo

- Acesso ao banco: somente o backend (Kotlin e Spring Boot) lê e grava no banco. O serviço em Python extrai texto, divide em trechos, gera embeddings e chama o modelo de linguagem, mas não acessa o banco: recebe e devolve os dados ao backend pela interface de programação (API) interna.
- Implementação: as chaves primárias são do tipo UUID (identificador universal único) gerado pelo backend; as alternativas das questões usam jsonb (tipo do PostgreSQL para dados JSON). Nas relações com chave composta, o backend mapeia as colunas como UUID simples, sem associações @ManyToOne de chave composta, e a integridade fica a cargo do banco. As tabelas de junção com atributos próprios (prova_assunto e prova_questao) serão entidades da Java Persistence API (JPA), e não relações @ManyToMany.
- Validação: o esquema SQL correspondente a este modelo (arquivo de migração V1__esquema_inicial.sql) foi executado em PostgreSQL 16 com pgvector e testado quanto ao isolamento entre usuários, às exclusões de material e de conta, às regras de tópicos com vários materiais, à detecção de questões repetidas, ao andamento das operações e ao agrupamento de provas.

Dicionário de dados

Regras que valem para todas as tabelas:

- Todas as tabelas possuem criado_em (timestamptz, obrigatória: data e hora de criação do registro, preenchida pelo banco) e atualizado_em (timestamptz, obrigatória: igual a criado_em na criação e renovada pelo banco, por gatilho, a cada alteração da linha).
- Todas as tabelas, exceto usuario, possuem usuario_id (uuid, obrigatória): dono do registro, chave estrangeira para usuario.
- Todas as tabelas, exceto as três de junção (questao_trecho, prova_assunto e prova_questao), possuem id (uuid, obrigatória): chave primária, gerada pelo backend; o banco não gera identificadores.
- Chaves estrangeiras entre tabelas de dados: em regra, são compostas com usuario_id (exemplo: questao (topico_id, usuario_id) referencia topico (id, usuario_id)), e cada tabela referenciada tem uma restrição de unicidade sobre (id, usuario_id). Assim, o banco impede que um registro referencie dados de outro usuário. Três famílias de chaves estrangeiras não incluem usuario_id e mesmo assim preservam o dono, porque a linha referenciada já pertence a um único usuário: (topico_limite_id, assunto_id) em prova_assunto, (prova_id, questao_id) em resposta e (material_id, ordem_trecho_inicial ou ordem_trecho_final) em topico_fonte.
- Exclusão: a chave estrangeira usuario_id de cada tabela usa ON DELETE CASCADE, de modo que excluir a conta remove todos os dados do usuário. trecho.material_id, topico_fonte.material_id, questao_trecho.questao_id e questao_trecho.trecho_id também usam ON DELETE CASCADE. Todas as demais chaves estrangeiras usam NO ACTION (impedem excluir um registro ainda referenciado; a verificação ocorre no fim da instrução, por isso a exclusão da conta não é bloqueada).
- Valores permitidos, intervalos numéricos e coerência entre colunas (por exemplo, status e datas de conclusão) são impostos por restrições CHECK do banco, e não apenas pelo backend.
- Isolamento por conta: todas as tabelas, exceto usuario, têm Row-Level Security (segurança em nível de linha do PostgreSQL) ativada, com política que só permite ler e gravar linhas cujo usuario_id seja o do usuário da sessão. A aplicação conecta com o papel nexo_app, que não é dono das tabelas nem ignora a segurança por linha; as migrações rodam com outro papel. Em cada transação, o backend executa SET LOCAL app.usuario_id com o identificador do usuário; sem isso, as consultas não retornam linhas e as gravações são recusadas.
- Índices: toda chave estrangeira e toda tabela, exceto usuario, têm índice iniciando na coluna correspondente (a segurança por linha filtra por usuario_id em toda consulta).
- Regras aplicadas pelo backend, e não pelo banco: converter o e-mail para minúsculas e gerar o hash Argon2id da senha antes de gravar; impedir a exclusão do assunto padrão "Geral" (o banco permite excluir um assunto vazio); garantir que a divisão proposta no UC05 cubra todo o material, sem lacunas (o banco só impede sobreposição e trechos inexistentes); escolher as questões de uma prova entre os assuntos e até o tópico-limite definidos (UC10); atribuir o mesmo grupo_id às provas separadas geradas no mesmo pedido (UC10); gravar a questão somente após a aprovação (UC09); descartar a questão repetida, isto é, de enunciado com o mesmo hash ou com similaridade acima do limiar configurado em relação às questões do mesmo tópico, calculando o hash sobre o enunciado normalizado (UC07); calcular o agendamento (UC12); conduzir as mudanças de status das operações e das provas; atualizar o progresso_percentual das operações a cada etapa; aplicar o limite de requisições por usuário contando registros de registro_geracao em uma janela de tempo; anular a referência ao

material nos registros de operação ao excluí-lo (as fontes dos tópicos saem em cascata); remover o arquivo do armazenamento depois da exclusão do material.

Nas tabelas a seguir, "Obrigatória" indica se a coluna aceita valor nulo (Não) ou não (Sim). Os tipos são os do PostgreSQL. Além das colunas listadas em cada tabela, valem as colunas e regras acima.

usuario — Pessoa que possui conta na plataforma.

Coluna	Tipo	Obrigatória	Descrição
nome	text	Sim	Nome de exibição do usuário.
email	text	Sim	E-mail de acesso. O banco exige que esteja em minúsculas.
senha_hash	text	Sim	Hash da senha (algoritmo Argon2id). A senha em texto puro nunca é armazenada.

- O e-mail é único. O id desta tabela é o identificador do dono usado em todas as demais tabelas (coluna usuario_id).

material — Arquivo de estudo enviado pelo usuário.

Coluna	Tipo	Obrigatória	Descrição
nome_arquivo	text	Sim	Nome original do arquivo.
tipo_arquivo	text	Sim	Tipo de mídia do arquivo (exemplo: application/pdf).
tamanho_bytes	bigint	Sim	Tamanho do arquivo em bytes (maior ou igual a 0).
quantidade_paginas	integer	Não	Número de páginas (maior ou igual a 1); nulo em formatos sem páginas, como texto simples.
hash_arquivo	text	Sim	Hash SHA-256 do conteúdo, usado para detectar envio repetido.
caminho_arquivo	text	Sim	Local de armazenamento do arquivo no servidor.
status_processamento	text	Sim	Valores permitidos: pendente, processando, pronto, falhou.

- Único: (usuario_id, hash_arquivo), apenas para materiais cujo status_processamento não é falhou. Assim, um arquivo que falhou pode ser reenviado.

trecho — Parte do texto de um material, com seu embedding (vetor numérico que representa o significado do texto), usada na busca por similaridade.

Coluna	Tipo	Obrigatória	Descrição
material_id	uuid	Sim	Material de origem.
ordem	integer	Sim	Posição do trecho no material, a partir de 1.
numero_pagina	integer	Não	Página de origem (maior ou igual a 1); nulo em formatos sem páginas.
tipo_conteudo	text	Sim	Valores permitidos: texto, tabela, figura.
conteudo	text	Sim	Em texto, o texto do trecho; em tabela, a tabela convertida em texto (Markdown); em figura, a descrição da imagem gerada por modelo de linguagem com visão (a imagem em si não é armazenada).
embedding	vector(1536)	Sim	Embedding do conteúdo. A dimensão (1536, no exemplo) é a do modelo de embedding adotado e é fixa na coluna.
modelo_embedding	text	Sim	Nome do modelo que gerou o embedding.

- Único: (material_id, ordem).
- Tabelas e figuras entram na mesma sequência de trechos, na posição em que aparecem no material; por isso as faixas dos tópicos também as incluem.
- A busca por similaridade é exata (sem índice aproximado) e sempre filtrada por material_id, que tem índice.
- Reindexar atualiza embedding e modelo_embedding no lugar. Os trechos não são apagados nem recriados, porque as faixas das fontes dos tópicos (topico_fonte) referenciam a ordem deles.

assunto — Matéria que agrupa tópicos.

Coluna	Tipo	Obrigatória	Descrição
nome	text	Sim	Nome do assunto.
assunto_padrao	boolean	Sim	Verdadeiro apenas para o assunto "Geral", criado junto com a conta.

- Único por usuário, sem diferenciar maiúsculas de minúsculas: (usuario_id, nome).
- No máximo um assunto com assunto_padrao verdadeiro por usuário.

topico — Unidade de estudo: um tema livre ou partes de um ou mais materiais (o vínculo com os materiais fica em topico_fonte).

Coluna	Tipo	Obrigatória	Descrição
assunto_id	uuid	Sim	Assunto ao qual o tópico pertence.
nome	text	Sim	Nome do tópico.
ordem	integer	Sim	Posição do tópico dentro do assunto, a partir de 1.

- Único: (assunto_id, ordem). A unicidade é conferida só no fim da transação, o que permite trocar ou deslocar a ordem de vários tópicos de uma vez.
- Tópico sem linhas em topico_fonte é um tópico livre (gerado pelo conhecimento geral do modelo) ou perdeu todos os seus materiais.

topico_fonte — Fonte de um tópico: a faixa de trechos de um material que alimenta o tópico. Um tópico pode ter várias fontes, de materiais diferentes.

Coluna	Tipo	Obrigatória	Descrição
topico_id	uuid	Sim	Tópico alimentado.
material_id	uuid	Sim	Material de origem.
ordem_trecho_inicial	integer	Sim	Ordem do primeiro trecho do material que pertence ao tópico (maior ou igual a 1).
ordem_trecho_final	integer	Sim	Ordem do último trecho do material que pertence ao tópico (maior ou igual ao inicial).

- Único: (topico_id, material_id): um tópico tem no máximo uma faixa por material.
- Chaves estrangeiras (material_id, ordem_trecho_inicial) e (material_id, ordem_trecho_final) para trecho (material_id, ordem): a faixa só pode apontar para trechos existentes do mesmo material.
- As faixas de um mesmo material não se sobrepõem, nem entre tópicos diferentes (restrição de exclusão, conferida no fim da transação, o que permite ajustar a fronteira entre dois tópicos em duas atualizações).
- material_id usa ON DELETE CASCADE: excluir o material remove as fontes que apontam para ele; o tópico permanece, sem essa fonte.

resumo — Resumo gerado para um tópico.

Coluna	Tipo	Obrigatória	Descrição
topico_id	uuid	Sim	Tópico resumido.
conteudo	text	Sim	Texto do resumo.

- Cada nova geração cria um novo registro; o mais recente (maior criado_em) é o exibido.

questao — Questão de múltipla escolha aprovada na verificação de fidelidade.

Coluna	Tipo	Obrigatória	Descrição
topico_id	uuid	Sim	Tópico ao qual a questão pertence.
enunciado	text	Sim	Texto da pergunta.
alternativas	jsonb	Sim	Lista ordenada com o texto de cada alternativa, de 2 a 5 itens (jsonb é o tipo do PostgreSQL para dados JSON).
resposta_correta	smallint	Sim	Posição da alternativa correta na lista, a partir de 0.
enunciado_hash	text	Sim	Hash SHA-256 (64 caracteres hexadecimais minúsculos) do enunciado normalizado (minúsculas, sem acentos, sem pontuação e com espaços únicos); detecta repetição exata.
embedding	vector(1536)	Sim	Embedding do enunciado, com a mesma dimensão do embedding dos trechos; detecta repetição por sentido.
modelo_embedding	text	Sim	Nome do modelo que gerou o embedding do enunciado.

- resposta_correta é maior ou igual a 0 e menor que o número de alternativas.
- Só são gravadas questões aprovadas e não repetidas; as rejeitadas não são armazenadas.
- Único: (topico_id, enunciado_hash): o banco impede duas questões de enunciado idêntico no mesmo tópico.
- Repetição por sentido (regra do backend, o banco não impõe): antes de gravar, o backend compara o embedding do novo enunciado com o das questões do mesmo tópico (busca exata, filtrada por topico_id) e descarta a questão se a similaridade (cosseno) atingir o limiar configurado (padrão sugerido: 0,90).
- Trocar o modelo de embedding exige reindexar também os enunciados das questões (embedding e modelo_embedding atualizados no lugar).

questao_trecho — Liga cada questão aos trechos que a sustentam (tabela de junção).

Coluna	Tipo	Obrigatória	Descrição
questao_id	uuid	Sim	Questão. Compõe a chave primária.
trecho_id	uuid	Sim	Trecho de apoio. Compõe a chave primária.

- Chave primária composta: (questao_id, trecho_id).
- Pode não haver linhas: questões de tópico livre ou cujo material foi excluído não têm trechos de apoio.

prova — Conjunto de questões montado pelo usuário.

Coluna	Tipo	Obrigatória	Descrição
quantidade_solicitada	integer	Sim	Quantidade de questões pedida (maior ou igual a 1). A quantidade efetiva é o número de linhas em prova_questao.
status	text	Sim	Valores permitidos: gerada, em_andamento, concluida.
concluida_em	timestamptz	Não	Data e hora da conclusão. É preenchida se, e somente se, o status é concluida.
grupo_id	uuid	Não	Identifica o pedido que gerou a prova. Provas separadas geradas no mesmo pedido têm o mesmo grupo_id (gerado pelo backend, sem tabela própria); nulo em prova única.

prova_assunto — Assuntos que compõem uma prova e o limite de estudo em cada um (tabela de junção).

Coluna	Tipo	Obrigatória	Descrição
prova_id	uuid	Sim	Prova. Compõe a chave primária.
assunto_id	uuid	Sim	Assunto. Compõe a chave primária.
topico_limite_id	uuid	Não	Último tópico (na ordem do assunto) incluído na prova. Nulo significa sem limite: todos os tópicos do assunto.

- Chave primária composta: (prova_id, assunto_id).
- Chave estrangeira (topico_limite_id, assunto_id) para topico (id, assunto_id): garante que o tópico-limite pertence ao assunto. Por isso, um tópico usado como limite não pode ser movido para outro assunto.

prova_questao — Questões que compõem uma prova (tabela de junção).

Coluna	Tipo	Obrigatória	Descrição
prova_id	uuid	Sim	Prova. Compõe a chave primária.
questao_id	uuid	Sim	Questão. Compõe a chave primária.
ordem	integer	Sim	Posição da questão na prova, a partir de 1.

- Chave primária composta: (prova_id, questao_id).
- Único: (prova_id, ordem), conferido só no fim da transação (permite inverter ou deslocar a ordem).

resposta — Resposta dada pelo usuário a uma questão.

Coluna	Tipo	Obrigatória	Descrição
questao_id	uuid	Sim	Questão respondida.
prova_id	uuid	Não	Prova em que foi respondida. Nulo em resposta avulsa ou de revisão.
alternativa_escolhida	smallint	Sim	Posição da alternativa escolhida, a partir de 0.
correta	boolean	Sim	Verdadeiro quando a alternativa escolhida é a correta; gravado no momento da resposta.
respondida_em	timestamptz	Sim	Data e hora da resposta.

- Chave estrangeira (prova_id, questao_id) para prova_questao: uma resposta dada em prova só pode ser de questão que faz parte da prova.

progresso_questao — Agendamento de revisão de uma questão para um usuário (variação do algoritmo SM-2, descrita no UC12).

Coluna	Tipo	Obrigatória	Descrição
questao_id	uuid	Sim	Questão agendada.
repeticoes	integer	Sim	Acertos consecutivos (maior ou igual a 0); zerado a cada erro. Inicial: 0.
intervalo_dias	integer	Sim	Dias entre a última e a próxima revisão, de 1 a 365. Inicial: 1.

Coluna	Tipo	Obrigatória	Descrição
fator_facilidade	numeric(3,2)	Sim	Multiplicador do intervalo. Inicial: 2,50; mínimo 1,30; máximo 2,50.
ultima_revisao	date	Sim	Data da última resposta que alterou o agendamento (fuso horário America/Sao_Paulo).
proxima_revisao	date	Sim	Data da próxima revisão: ultima_revisao mais intervalo_dias.

- Único: (usuario_id, questao_id). O registro é criado na primeira resposta à questão.
- proxima_revisao é maior ou igual a ultima_revisao.
- A prioridade do tópico não é gravada: é calculada na consulta (questões com revisão vencida, depois taxa de erro).

registro_geracao — Registro de cada operação que usa o modelo de linguagem ou de embedding. Serve à auditoria, ao custo, à taxa de rejeição, ao limite de requisições por usuário e ao acompanhamento do andamento da operação.

Coluna	Tipo	Obrigatória	Descrição
topico_id	uuid	Não	Tópico a que a operação se refere. Nulo quando ainda não há tópico (indexacao_material e sugestao_topicos).
material_id	uuid	Não	Material a que a operação se refere. Preenchido em indexacao_material e sugestao_topicos, que ainda não têm tópico; nulo nas demais. Passa a nulo se o material for excluído.
operacao	text	Sim	Valores permitidos: indexacao_material, sugestao_topicos, resumo, questoes, verificacao_questao.
tipo_entrada	text	Sim	Valores permitidos: material, topico_livre.
modelo_ia	text	Sim	Nome do modelo de linguagem ou de embedding utilizado.
status	text	Sim	Valores permitidos: pendente, em_andamento, concluida, falhou.
tokens_entrada	integer	Sim	Tokens enviados ao modelo (maior ou igual a 0; 0 até a conclusão).
tokens_saida	integer	Sim	Tokens recebidos do modelo (maior ou igual a 0; 0 até a conclusão).
custo_estimado_usd	numeric(10,6)	Sim	Custo estimado, em dólares americanos (maior ou igual a 0).
questoes_aprovadas	integer	Sim	Questões aprovadas na verificação (preenchido na operação verificacao_questao; 0 nas demais).
questoes_rejeitadas	integer	Sim	Questões reprovadas na verificação (preenchido na operação verificacao_questao; 0 nas demais).
questoes_repetidas	integer	Sim	Questões descartadas por repetição (preenchido na operação questoes; 0 nas demais).
progresso_percentual	smallint	Sim	Andamento da operação, de 0 a 100; é 100 quando o status é concluida. Atualizado pelo backend a cada etapa e consultado pelo frontend para exibir a barra de progresso.
finalizada_em	timestamptz	Não	Data e hora do término. É preenchida se, e somente se, o status é concluida ou falhou.

- Cada operação tem o seu próprio registro. Os tokens e o custo das chamadas feitas dentro de uma operação (por exemplo, obter o conteúdo-base) são somados ao registro da operação que as chamou.
- material_id é opcional e, quando preenchido, forma com usuario_id uma chave estrangeira para material (id, usuario_id). Na exclusão do material, o backend anula essa coluna (UC17); assim, o registro da operação, com seus tokens e custo, é mantido.
- progresso_percentual: na indexacao_material, o backend calcula o percentual por etapas (extração, divisão em trechos, embeddings em lotes e gravação) e o frontend consulta o registro a cada poucos segundos; nas demais operações pode ser calculado pelo número de itens concluídos.
- Taxa de rejeição = soma de questoes_rejeitadas dividida pela soma de (questoes_aprovadas + questoes_rejeitadas), considerando apenas registros da operação verificacao_questao.

3. Arquitetura do Sistema

A Figura 4 mostra a arquitetura da Plataforma Nexo e o sentido das chamadas entre os componentes. O estudante usa o frontend em React, que acessa o sistema por HTTPS. O servidor virtual privado (VPS) hospeda o Caddy, que atua como proxy reverso, cuida do certificado HTTPS e encaminha as requisições ao backend, e os três serviços containerizados com Docker Compose: o backend, o serviço de processamento e o banco de dados. Fora do servidor fica apenas o provedor do modelo de linguagem, uma API externa (por exemplo, Gemini ou Claude).

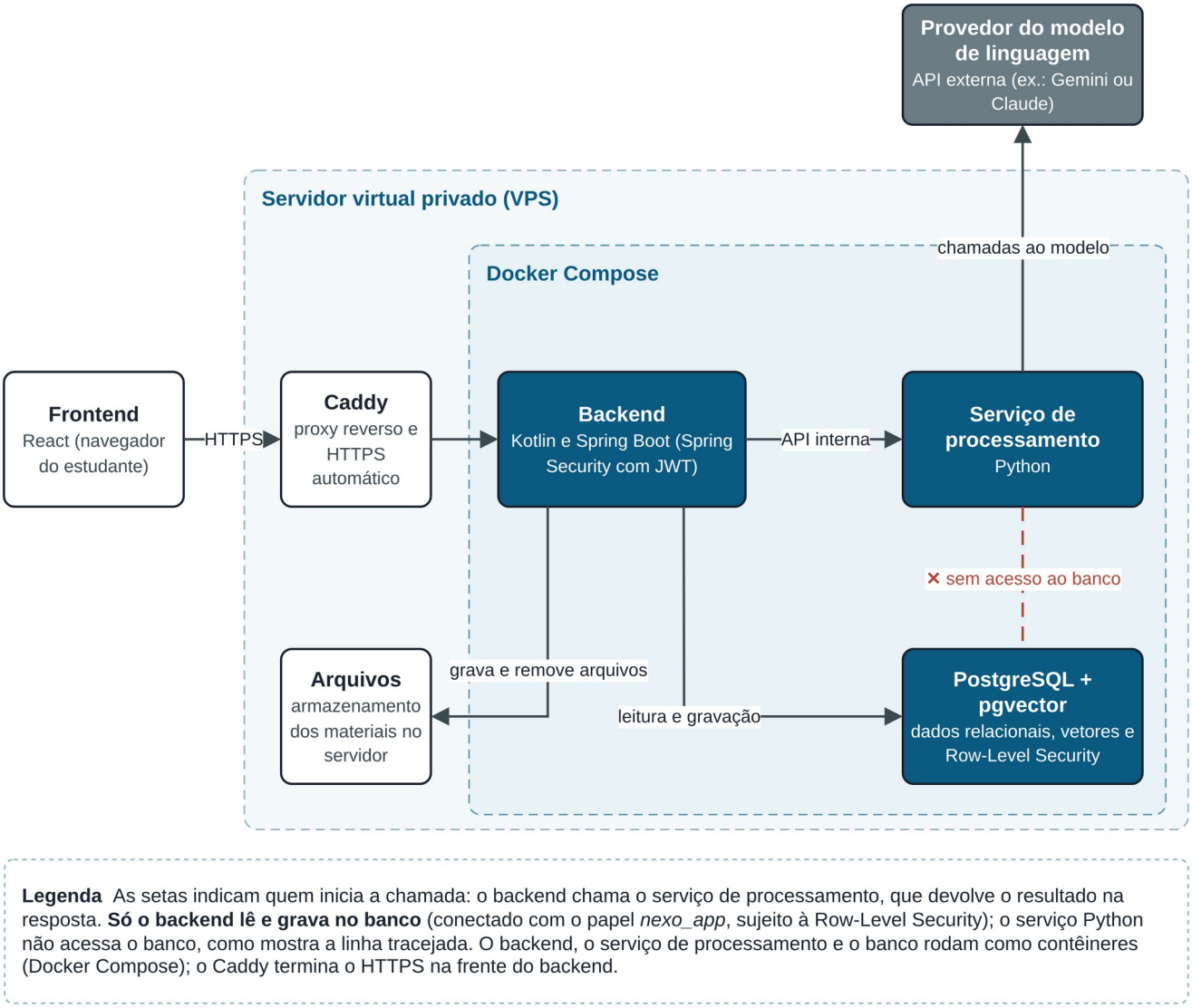


Figura 4 – Arquitetura da Plataforma Nexo: componentes e sentido das chamadas

O backend (Kotlin e Spring Boot) concentra as regras de negócio e a autenticação com JWT e é o único componente que lê e grava no PostgreSQL com pgvector, onde o isolamento por conta é garantido por Row-Level Security; é também ele quem grava e remove os arquivos enviados. O serviço de processamento (Python) é chamado pelo backend por uma API interna: extrai o texto, as tabelas e as figuras, divide em trechos, gera os embeddings e chama o modelo de linguagem, devolvendo o resultado ao backend, que o grava. Esse serviço não acessa o banco de dados; como só o backend o acessa, as regras de acesso e de exclusão (UC17 e UC18) ficam concentradas nele.

4. Ferramentas Previstas para o Desenvolvimento do Projeto

- **Kotlin e Spring Boot —** API principal (backend): regras de negócio, autenticação (Spring Security com JWT, JSON Web Token) e orquestração das chamadas ao serviço de processamento. É o único componente que acessa o banco de dados.
- **Python —** Serviço de processamento: extração de texto, tabelas e figuras, divisão em trechos, geração de embeddings e chamadas ao modelo de linguagem (inclusive o modelo com visão que descreve figuras). Não acessa o banco de dados.

- **PostgreSQL e pgvector —** Banco de dados único: dados relacionais (usuários, materiais, questões, provas, respostas) e busca por similaridade de vetores, sem um banco vetorial separado. O isolamento por conta é reforçado com Row-Level Security.
- **Flyway —** Versionamento das migrações do esquema do banco de dados.
- **React —** Frontend responsivo: envio de materiais, geração de conteúdo, resolução de questões e painel de desempenho.
- **API de modelo de linguagem (por exemplo, Gemini ou Claude) —** Geração de resumos e questões, sugestão de divisão em tópicos, verificação de fidelidade das questões, descrição de figuras e geração de embeddings (dos trechos e dos enunciados das questões).
- **Docker e Docker Compose —** Containerização dos serviços (backend, serviço de processamento e banco), padronizando o ambiente e simplificando a implantação.
- **Servidor virtual privado (VPS) com Caddy —** Hospedagem com custo fixo e previsível, proxy reverso e certificado HTTPS (protocolo HTTP seguro) automático.
- **Git e GitHub —** Controle de versão e histórico do desenvolvimento.
- **IntelliJ IDEA e Visual Studio Code —** Ambientes de desenvolvimento do backend (Kotlin) e do serviço de processamento e frontend (Python e React).
- **draw.io (diagrams.net) —** Modelagem dos diagramas de casos de uso e entidade-relacionamento.

5. Cronograma

A tabela a seguir distribui as etapas do projeto em oito semanas (S1 a S8), com a entrega da AS ao final da S8. A ordem segue a dependência entre as entregas: o levantamento de requisitos e a análise ocupam a S1 e a S2, e o projeto fecha na S2 com os diagramas e o modelo de dados, para que o desenvolvimento (S3 a S6) comece com o esquema do banco definido. Os testes começam na S5, quando as primeiras funcionalidades ficam prontas, e acompanham o desenvolvimento até a S8, semana em que ocorre também a implantação.

Etapa	S1	S2	S3	S4	S5	S6	S7	S8
Levantamento Requisitos	X							
Análise	X	X						
Projeto		X						
Desenvolvimento			X	X	X	X		
Testes					X	X	X	X
Implantação								X